QUY TRÌNH PHÁT TRIỂN CODE DỰ ÁN MCU

1. Cấu Trúc Nhánh (Branch Strategy)

- `main`: Code chạy thật trên kit, hoàn toàn ổn định (Production). Cấm commit trực tiếp.
- `develop`: Nhánh tích hợp chính. Tất cả nhánh con tạo từ đây và merge về đây.
- `feature/<tên-task>`: Nhánh làm việc riêng của từng cá nhân.
- `integration/<phiên-bản>`: Nhánh ghép nối logic hệ thống trước khi release.

---

2. Giai Đoạn 1: Setup Repository & Xây Dựng HAL

Bước 1: Trưởng nhóm (Leader) tạo Repo và khung thư mục

1. Khởi tạo Git Repo trên GitHub / GitLab.
2. Thêm file `.gitignore` cho dự án MCU (bỏ file build `*.o`, `*.elf`, `*.hex`, `build/`, file cấu hình IDE cá nhân).
3. Commit cấu trúc thư mục chuẩn (đã tích hợp FreeRTOS):

```
mcu-project/
├── bsp/              # Board support & Pin mapping
├── hal/              # Drivers ngoại vi (Hardware Abstraction Layer)
│   ├── inc/          # hal_gpio.h, hal_uart.h, hal_i2c.h...
│   └── src/          # hal_gpio.c, hal_uart.c, hal_i2c.c...
├── middlewares/      # FreeRTOS source (Kernel, Port, Heap)
├── app/              # FreeRTOS Tasks & IPC
│   ├── inc/          # task_sensor.h, task_display.h, app_ipc.h...
│   └── src/          # task_sensor.c, task_display.c, app_ipc.c...
├── main.c            # Init hardware, tạo task, vTaskStartScheduler()
├── .gitignore
└── README.md
```

4. Push code khung lên `main`, sau đó tạo nhánh `develop`:

```
git branch -M main
git push -u origin main
git checkout -b develop
git push -u origin develop
```

Bước 2: Thống nhất API Interface của HAL & Cơ chế IPC

Trước khi code driver, Leader định nghĩa trước:

- Header file mẫu trong `hal/inc/`: đảm bảo các hàm HAL **Thread-Safe** (sử dụng Mutex hoặc ngắt/DMA đồng bộ với FreeRTOS API `FromISR`).
- Header IPC dùng chung (`app/inc/app_ipc.h`): Khai báo các Queue Handles, Event Groups hoặc Mutex để các Task trao đổi dữ liệu.

Bước 3: Phân chia và code HAL song song

- **Dev 1:** Phụ trách `feature/hal-gpio-uart`
- **Dev 2:** Phụ trách `feature/hal-i2c-spi`
- **Dev 3:** Phụ trách `feature/hal-timer-adc`

Mỗi Dev thực hiện:

```
# Lấy code mới nhất và tạo branch
git checkout develop
git pull origin develop
git checkout -b feature/hal-gpio-uart

# Viết code, test driver trên board, commit
git add hal/
git commit -m "feat(hal): implement basic uart transmission"

# Đẩy lên remote
git push origin feature/hal-gpio-uart
```

- Mở Pull Request (PR) từ branch của mình vào `develop`.
- Các thành viên còn lại review: kiểm tra đúng API thống nhất, không dùng hàm block vĩnh viễn không timeout, không gây dead-lock với RTOS.
- Leader duyệt và Merge lần lượt vào `develop`.

---

3. Giai Đoạn 2: Phát Triển Task FreeRTOS Độc Lập

Khi nhánh `develop` đã hoàn tất HAL và cấu hình xong FreeRTOS Kernel:

Bước 1: Thống nhất tài nguyên RTOS

Trước khi code, nhóm họp thống nhất bảng thông số trong `app/inc/app_cfg.h`:

- **Task Priority**: Task nào ưu tiên cao nhất (ví dụ: `TASK_COMM_PRIO` > `TASK_SENSOR_PRIO` > `TASK_DISPLAY_PRIO`).
- **Stack Size**: Dung lượng stack cấp cho từng task (ví dụ: 256 words cho Sensor, 512 words cho Display/Comm).
- **Queues**: Kiểu dữ liệu đóng gói gửi qua `xQueueSend()` / `xQueueReceive()`.

Bước 2: Nhận Task và tạo nhánh mới

- **Dev 1 (Cảm biến):** `feature/task-sensor`
- **Dev 2 (Màn hình/UI):** `feature/task-display`
- **Dev 3 (Truyền thông/Network):** `feature/task-comm`

```
git checkout develop
git pull origin develop
git checkout -b feature/task-sensor
```

Bước 3: Quy chuẩn triển khai một Task File

Mỗi task nằm gọn trong `app/src/task_xxx.c` và cung cấp đúng hàm khởi tạo Task ra ngoài `app/inc/task_xxx.h`:

```
/* task_sensor.c */
#include "FreeRTOS.h"
#include "task.h"
#include "task_sensor.h"
#include "app_ipc.h"

static TaskHandle_t xSensorTaskHandle = NULL;

static void prvSensorTask(void *pvParameters) {
    /* Khởi tạo ngoại vi riêng của task nếu cần */
    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xPeriod = pdMS_TO_TICKS(100); /* Chu kỳ 100ms */

    for (;;) {
        /* Đọc cảm biến qua HAL */
        // SensorData_t data = read_sensor();

        /* Gửi data vào Queue chung bằng FreeRTOS API */
        // xQueueSend(xSensorQueue, &data, 0);

        vTaskDelayUntil(&xLastWakeTime, xPeriod);
    }
}

BaseType_t task_sensor_create(void) {
    return xTaskCreate(
        prvSensorTask,
        "SENSOR_TASK",
        configMINIMAL_STACK_SIZE + 128,
        NULL,
        tskIDLE_PRIORITY + 2,
        &xSensorTaskHandle
    );
}
```

- Tuyệt đối không can thiệp code vào `main.c`.
- Sử dụng mock queue hoặc dummy data cục bộ để test độc lập trên kit trước khi ghép.

Bước 4: Đồng bộ liên tục với `develop`

```
git checkout feature/task-sensor
git fetch origin
git rebase origin/develop

# Nếu có xung đột (Conflict):
# 1. Mở file conflict, sửa lại đúng
# 2. git add <file_da_sua>
# 3. git rebase --continue

git push --force-with-lease origin feature/task-sensor
```

Bước 5: Mở Pull Request và Merge

- Mở PR `feature/task-xxx` -> `develop`.
- Yêu cầu kiểm tra: Task tạo thành công, không tràn Stack (`vApplicationStackOverflowHook`), biên dịch không cảnh báo.
- Review và Merge vào `develop`.

---

4. Giai Đoạn 3: Tích Hợp Toàn Bộ Hệ Thống (Integration)

Khi cả 3 task đã được merge vào `develop`:

Bước 1: Tạo nhánh tích hợp

```
git checkout develop
git pull origin develop
git checkout -b integration/system-v1
```

Bước 2: Ghép nối tại `main.c` và Khởi chạy Scheduler

1 người đại diện (hoặc cả 3 cùng làm việc) hoàn thiện `app_ipc.c` và `main.c`:

1. **Khởi tạo cơ chế IPC (`app_ipc.c`):**
   - Tạo các Queue, Mutex, Semaphore dùng chung.
2. **Khởi tạo và Start Scheduler trong `main.c`:**

```c
int main(void) {
    /* 1. Hardware & HAL init */
    bsp_system_init();
    hal_gpio_init();
    hal_uart_init();

    /* 2. FreeRTOS IPC Object Creation */
    app_ipc_init(); /* Tạo xSensorQueue, xCommMutex... */

    /* 3. Create Tasks */
    if (task_sensor_create()  != pdPASS) { /* Handle error */ }
    if (task_display_create() != pdPASS) { /* Handle error */ }
```

```
    if (task_comm_create()    != pdPASS) { /* Handle error */ }

    /* 4. Start FreeRTOS Scheduler */
    vTaskStartScheduler();

    /* Chương trình không bao giờ chạy đến đây nếu RTOS đủ heap */
    for (;;);
}
```

3. **Kiểm tra và hiệu chỉnh hệ thống:**
   - Kiểm tra Heap khả dụng: `xPortGetFreeHeapSize()`.
   - Kiểm tra High Water Mark (mức tiêu thụ stack đỉnh) của từng task: `uxTaskGetStackHighWaterMark()`.
   - Xử lý độ ưu tiên (Priority Inversion, Deadlock nếu có).
   - Commit và sửa các lỗi tích hợp trực tiếp trên nhánh `integration/system-v1`.

Bước 3: Đóng gói và phát hành (Release)

Sau khi hệ thống đa nhiệm chạy ổn định trên phần cứng:

```
# 1. Merge vào develop
git checkout develop
git merge --no-ff integration/system-v1
git push origin develop

# 2. Merge vào main và đánh tag
git checkout main
git merge --no-ff develop
git tag -a v1.0.0 -m "Release v1.0.0: FreeRTOS integrated system stable"
git push origin main --tags

# 3. Xóa nhánh integration đã hoàn tất
git branch -d integration/system-v1
```

---

5. Quy Chuẩn Kỷ Luật Dự Án (Bắt Buộc Tuân Thủ)

1. **Về IDE Code Generation (CubeMX, MCUXpresso, MPLAB...):**
   - Chỉ duy nhất **01 người phụ trách** cấu hình chân, clock, middleware FreeRTOS và generate code.
   - Cấm mỗi người tự mở bộ cấu hình sinh code đè lên cấu hình FreeRTOS chung (`FreeRTOSConfig.h`).
2. **Nguyên tắc an toàn RTOS:**
   - Không gọi API thường trong hàm ngắt ISR (chỉ dùng các hàm có đuôi `...FromISR()`).
   - Không dùng các hàm delay phần cứng thô (`delay_ms()`, `HAL_Delay()`) trong task; bắt buộc dùng `vTaskDelay()` hoặc `vTaskDelayUntil()`.

3. **Quy tắc Commit Message:**
    - `feat(task-xxx)`: Thêm chức năng task hoặc IPC mới.
    - `fix(rtos)`: Sửa lỗi stack, priority, deadlock hoặc lỗi HAL driver.
    - `refactor(...)`: Tối ưu hóa cấu trúc code.
4. **Không commit file rác:**
    - Tuyệt đối không commit thư mục `build/`, `Debug/`, `Release/` và các file tạm của IDE.